Reviewer Pack — Fourier Norm Lower Bound for Disjointness Communication Matr...

Entry af4613884bf0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 03:12:08 UTC

Fourier Norm Lower Bound for Disjointness Communication Matrices

Entry ID: `af4613884bf0` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-21 03:12:08 UTC

1. Conjecture

Field A (mathematical branch): Harmonic Analysis on Locally Compact Groups **Field B** (complexity object): Randomized Communication Complexity of Disjointness

Statement:

> For any n×n communication matrix M of the Disjointness problem, the L^2 norm of its Fourier transform over the additive group Z_n^2 is Ω(n).

Rationale (proposer's reasoning):

> Harmonic analysis on finite abelian groups captures the spectral structure of communication matrices, with Fourier norms reflecting the 'spread' of information across frequencies. The Disjointness matrix's inherent combinatorial structure may enforce nontrivial Fourier norms, linking algebraic invariants to communication complexity.

Taxonomy category: `ACC_LB_via_WILLIAMS` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `87f86b322d8f3072`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    n = 40
    random.seed(seed)

    def generate_disjointness_matrix(n):
        M = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(i + 1, n):
                M[i][j] = 1
                M[j][i] = 1
        return M

    def fft(a):
        n = len(a)
        if n == 1:
            return a
        even = fft([a[k] for k in range(0, n, 2)])
        odd = fft([a[k] for k in range(1, n, 2)])
        T = [cmath.exp(-2j * math.pi * k / n) * odd[k] for k in range(n // 2)]
        return [even[k] + T[k] for k in range(n // 2)] + [even[k] - T[k] for k in range(n // 2)]

    def fft_2d(M):
        n = len(M)
        M_fft = [[0] * n for _ in range(n)]
        for i in range(n):
            M_fft[i] = fft([M[i][j] for j in range(n)])
        return [fft(row) for row in M_fft]

    def l2_norm(matrix):
        sum_of_squares = 0
        for row in matrix:
            for val in row:
                sum_of_squares += abs(val) ** 2
        return math.sqrt(sum_of_squares)

    M = generate_disjointness_matrix(n)
```

```
    F = fft_2d(M)
    norm = l2_norm(F)

    return {
        "metric_name": "Fourier Norm",
        "metric_value": norm,
        "instances_tested": 1,
        "conjecture_holds": norm >= n,
        "counterexample": "" if norm >= n else "norm < n"
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or list(range(2, 30))  # Default to first 29 primes
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_norm = sum(result["metric_value"] for result in results) / len(results)
    std_norm = math.sqrt(sum((result["metric_value"] - mean_norm) ** 2 for result in results) / le
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_norm} std={std_norm} support_fraction={support_fracti
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_norm} std={std_norm} support_fraction={support_fracti
    else:
        for result in results:
            if not result["conjecture_holds"]:
                print(f"RESULT: FALSIFIED counterexample=\"norm < n\" first_failing_seed={result['
                break
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3a7c6574.py", line 70, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3a7c6574.py", line 54, in run_trial
    F = fft_2d(M)
        ^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3a7c6574.py", line 43, in fft_2d
    M_fft[i] = fft([M[i][j] for j in range(n)])
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3a7c6574.py", line 34, in fft
    even = fft([a[k] for k in range(0, n, 2)])
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3a7c6574.py", line 34, in fft
```

```
    even = fft([a[k] for k in range(0, n, 2)])
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3a7c6574.py", line 34, in fft
    even = fft([a[k] for k in range(0, n, 2)])
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   [Previous line repeated 2 more times]
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3a7c6574.py", line 36, in fft
    T = [cmath.exp(-2j * math.pi * k / n) * odd[k] for k in range(n // 2)]
         ^^^^^
NameError: name 'cmath' is not defined. Did you mean: 'math'? Or did you forget to import 'cmath'?
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to missing cmath import, preventing evaluation of Fourier norm bounds. | next: Fix the test by importing cmath and re-running with proper error handling

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	98190
2	propose	ollama_remote	qwen3:8b	0	0	138323
3	propose	ollama_remote	qwen3:8b	0	0	102310
4	propose	ollama_remote	qwen3:8b	0	0	124210
5	preregistration	ollama_remote	qwen3:8b	0	0	31664
6	novelty	ollama_remote	qwen3:8b	0	0	27835
7	novelty	ollama_remote	qwen3:8b	0	0	23925
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14537
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10894
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9815
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8742
12	judge	ollama_remote	qwen3:8b	0	0	20910

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 611356 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/af4613884bf0.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/af4613884bf0.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/af4613884bf0.tar.gz` (if generated)